El Perceptrón

Licenciatura en Matemáticas Aplicadas y Computación

Facultad de Estudios Superiores Acatlán
Universidad Nacional Autónoma de México



Contenido

1. Regresión lineal y descenso de gradiente.
2. La neurona artificial: suma ponderada y activación.
3. El perceptrón: frontera de decisión y regla de aprendizaje.
4. Ejemplos: compuertas lógicas AND y OR.
5. Limitación del modelo: XOR y perceptrón multicapa.
6. Implementación en Python: NumPy y scikit-learn.

Regresión lineal

Datos: $\{(x^{(i)}, y^{(i)})\}_{i=1}^{m}$, con $x^{(i)} \in \mathbb{R}$ y $y^{(i)} \in \mathbb{R}$.

Hipótesis

$$h_\theta(x) = \theta_0 + \theta_1 x$$

Función de costo (error cuadrático medio)

$$J(\theta_0, \theta_1) = \frac{1}{2m} \sum_{i=1}^{m} \left(h_\theta(x^{(i)}) - y^{(i)}\right)^2$$

El **cuadrado** evita que los errores se cancelen y penaliza los errores grandes. J es **convexa** (un único mínimo global) y el factor $\frac{1}{2}$ simplifica la derivada.

Descenso de gradiente

Derivadas parciales

$$\frac{\partial J}{\partial \theta_0} = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right), \qquad \frac{\partial J}{\partial \theta_1} = \frac{1}{m}\sum_{i=1}^{m}\left(h_\theta(x^{(i)}) - y^{(i)}\right) x^{(i)}$$

El gradiente ∇J apunta hacia donde J **crece**. Para minimizar, avanzamos en sentido opuesto:

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial J}{\partial \theta_j}, \qquad j = 0, 1.$$

$\alpha > 0$ es la **tasa de aprendizaje**. Cada paso utiliza *todos* los datos.

Ejemplo: un paso de descenso de gradiente

i	$x^{(i)}$	$y^{(i)}$
1	1	2
2	2	5
3	3	5

Partimos de $\theta_0 = \theta_1 = 0$ y tomamos $\alpha = 0.1$.

Predicciones iniciales $h_\theta = (0, 0, 0)$, luego los errores son $h_\theta - y = (-2, -5, -5)$.

$$\frac{\partial J}{\partial \theta_0} = \frac{-2 - 5 - 5}{3} = -4$$

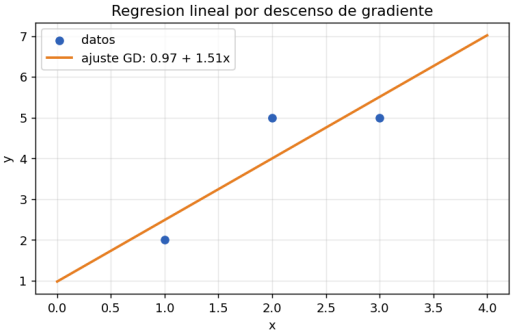
$$\frac{\partial J}{\partial \theta_1} = \frac{(-2)(1) + (-5)(2) + (-5)(3)}{3} = -9$$

$$\theta_0 = 0 - 0.1(-4) = 0.4, \qquad \theta_1 = 0 - 0.1(-9) = 0.9.$$

Convergencia

Paso	θ_0	θ_1	J
0	0.000	0.000	9.0000
1	0.400	0.900	1.9900
2	0.580	1.300	0.5995
3	0.662	1.477	0.3236
4	0.700	1.556	0.2688
∞	1.000	1.500	0.2500

La última fila es la solución de la **ecuación normal** $\theta = (X^T X)^{-1} X^T y$.



Tasa de aprendizaje α

- α pequeña: convergencia lenta.
- α adecuada: descenso rápido y estable.
- α grande: el costo oscila y puede divergir.

En la práctica α se elige por experimentación. Graficar J contra las iteraciones es el diagnóstico estándar.

De la recta a la neurona

La regresión lineal puede escribirse como una sola función compuesta:

$$h(x) = g(z), \qquad z = w_0 + w_1 x.$$

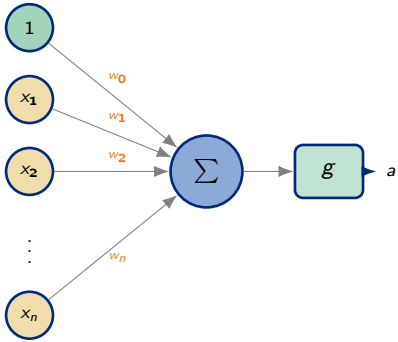
- Si g es la **identidad**, h es la regresión lineal.
- Si g es un **escalón**, h es un clasificador binario.

Los parámetros (θ_0, θ_1) se renombran (w_0, w_1) y se denominan **pesos**.

Neurona artificial

$$z = w_0 + \sum_{j=1}^{n} w_j x_j = \mathrm{w}^\top \mathrm{x}, \qquad a = g(z).$$

- x_j: entradas (*features*).
- w_j: pesos; miden la importancia de cada entrada.
- w_0: **sesgo**, con entrada constante $x_0 = 1$.
- g: **función de activación**.



Función de activación

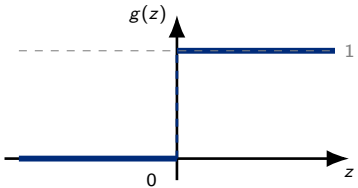
La composición de funciones lineales es lineal. Sin una activación no lineal, apilar neuronas no aumenta la capacidad del modelo.

Activación	$g(z)$	Uso típico
Identidad	z	regresión
Escalón	$\begin{cases} 1, & z > 0 \\ 0, & z \leq 0 \end{cases}$	perceptrón
Sigmoide	$\frac{1}{1+e^{-z}}$	clasificación suave
ReLU	$\max(0, z)$	redes profundas

La función escalón

$$g(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0. \end{cases}$$

- Salida binaria: el signo de la evidencia z.
- El umbral se fija en 0 porque el sesgo w_0 lo puede desplazar.
- No es diferenciable en $z = 0$; por ello el perceptrón no se entrena con el gradiente del MSE.

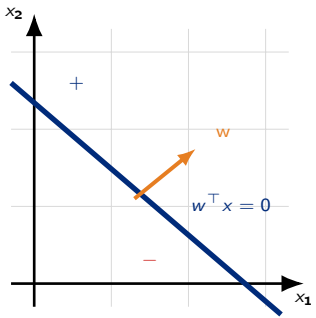


Frontera de decisión

La ecuación $\mathrm{w}^\top \mathrm{x} = 0$ define un **hiperplano** que divide el espacio en dos regiones. En el plano ($n = 2$):

$$w_0 + w_1 x_1 + w_2 x_2 = 0 \implies x_2 = -\frac{w_0 + w_1 x_1}{w_2}.$$

- $\mathrm{w} = (w_1, w_2)$ es perpendicular a la frontera y apunta hacia la clase 1.
- El sesgo w_0 traslada la frontera paralelamente.



El perceptrón

Clasificador binario con activación escalón:

$$\hat{y} = g(\mathrm{w}^\top \mathrm{x}), \qquad g(z) = \begin{cases} 1, & z > 0 \\ 0, & z \leq 0. \end{cases}$$

- $\mathrm{w}^\top \mathrm{x} > 0 \Rightarrow$ clase 1; $\mathrm{w}^\top \mathrm{x} \leq 0 \Rightarrow$ clase 0.
- La frontera es el hiperplano $\mathrm{w}^\top \mathrm{x} = 0$.
- Sin sesgo, la frontera pasa por el origen; w_0 lo evita.

¿Por qué no usar el Error Cuadrático Medio?

Con activación escalón, el gradiente del ECM es 0 en casi todas partes, así que no sirve para ajustar los pesos. En su lugar se usa el **Criterio del Perceptrón**.

Función de pérdida

$$L(\mathrm{w}) = \max\bigl(0,\ -y'\,\mathrm{w}^{\top}\mathrm{x}\bigr)$$

Regla de actualización (descenso de gradiente)

$$\mathrm{w} \leftarrow \mathrm{w} + \alpha\, y'\,\mathrm{x}.$$

Convergencia (Novikoff, 1962)

Si los datos son separables por una recta, el algoritmo converge en pasos finitos.

Algoritmo

1. Inicializar $\mathrm{w} = 0$.
2. Por cada época, recorrer los ejemplos y actualizar cuando $\hat{y} \neq y$.
3. Detener cuando una época completa no produzca errores.

Sesgo como peso adicional

Se añade la coordenada $x_0 = 1$ y se trata w_0 como un peso más: $z = \mathrm{w}^\top \mathrm{x}$.

Conviene barajar el orden de los ejemplos en cada época.

Compuertas lógicas

x_1	x_2	AND	OR	XOR
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Usamos la entrada de sesgo $x_0 = 1$, de modo que $z = w_0 + w_1 x_1 + w_2 x_2$. Inicializamos $\mathrm{w} = (0, 0, 0)$ y tomamos $\alpha = 1$.

OR: época 1

x $= (1, x_1, x_2)$	y	z	$\hat{y}$	acción	w
$(1, 0, 0)$	0	0	0	—	$(0, 0, 0)$
$(1, 0, 1)$	1	0	0	+x	$(1, 0, 1)$
$(1, 1, 0)$	1	1	1	—	$(1, 0, 1)$
$(1, 1, 1)$	1	2	1	—	$(1, 0, 1)$

Con w $= (0, 0, 0)$ todos los puntos caen del lado $\hat{y} = 0$. El primer error $(y = 1, \hat{y} = 0)$ suma x para aumentar z.

OR: épocas 2 y 3

Época	x	y	z	$\hat{y}$	w	
2	$(1,0,0)$	0	1	1	$(0,0,1)$	$-$x
2	$(1,0,1)$	1	1	1	$(0,0,1)$	
2	$(1,1,0)$	1	0	0	$(1,1,1)$	$+$x
2	$(1,1,1)$	1	3	1	$(1,1,1)$	
3	$(1,0,0)$	0	1	1	$(0,1,1)$	$-$x
3	$(1,0,1)$	1	1	1	$(0,1,1)$	
3	$(1,1,0)$	1	1	1	$(0,1,1)$	
3	$(1,1,1)$	1	2	1	$(0,1,1)$	

La época 4 no tiene errores: la frontera es $x_1 + x_2 = 0$ y clasifica OR correctamente.

AND

Entrenar a mano el perceptrón para la compuerta AND con $\mathrm{w} = (0, 0, 0)$ y $\alpha = 1$. Construir la tabla de actualizaciones hasta que una época no tenga errores.

x_1	x_2	AND
0	0	0
0	1	0
1	0	0
1	1	1

La solución converge en la época 6 a $\mathrm{w} = (-2, 2, 1)$:

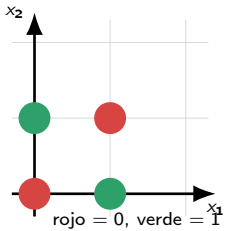
x	y	z	$\hat{y}$
$(1, 0, 0)$	0	-2	0
$(1, 0, 1)$	0	-1	0
$(1, 1, 0)$	0	0	0
$(1, 1, 1)$	1	1	1

La frontera es $x_2 = 2 - 2x_1$. Los pesos no son únicos.

XOR: un problema no separable

x_1	x_2	XOR
0	0	0
0	1	1
1	0	1
1	1	0

Queremos 1 cuando las entradas son distintas y 0 cuando son iguales.



Demostración de no separabilidad

Supongamos que un perceptrón $z = w_0 + w_1 x_1 + w_2 x_2$ clasifica bien XOR. Evaluemos cada caso de la tabla de verdad:

x_1	x_2	XOR	Condición de z	Inecuación resultante
0	0	0	$w_0 + w_1(0) + w_2(0) \leq 0$	**(1)** $w_0 \leq 0$
1	0	1	$w_0 + w_1(1) + w_2(0) > 0$	**(2)** $w_0 + w_1 > 0$
0	1	1	$w_0 + w_1(0) + w_2(1) > 0$	**(3)** $w_0 + w_2 > 0$
1	1	0	$w_0 + w_1(1) + w_2(1) \leq 0$	**(4)** $w_0 + w_1 + w_2 \leq 0$

Si sumamos las inecuaciones (2) y (3) obtenemos:

$$2w_0 + w_1 + w_2 > 0$$

Pero si sumamos las inecuaciones (1) y (4) obtenemos:

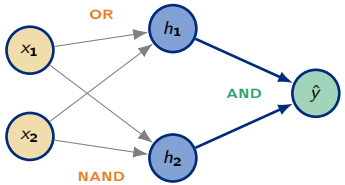
$$2w_0 + w_1 + w_2 \leq 0$$

Capas ocultas

XOR = OR AND NAND.

- Una neurona oculta aprende OR.
- Otra neurona oculta aprende NAND.
- La neurona de salida combina ambas con AND.

La capa oculta transforma el espacio a uno donde XOR sí es separable.



Perceptrón multicapa

- **Capa de entrada:** recibe las *features*.
- **Capas ocultas:** aplican transformaciones no lineales.
- **Capa de salida:** produce la predicción.

El entrenamiento de varias capas se realiza con **retropropagación** (*backpropagation*): la regla de la cadena aplicada al gradiente de la pérdida. Es la misma idea de reducir el error, ahora en todos los pesos de la red.

Regresión lineal con NumPy

```python
import numpy as np

X = np.array([1., 2., 3.])
y = np.array([2., 5., 5.])

def gradiente(t0, t1, X, y):
    error = (t0 + t1 * X) - y           # h(x) - y
    return error.mean(), (error * X).mean()

t0, t1, alpha = 0.0, 0.0, 0.1
for _ in range(200):
    g0, g1 = gradiente(t0, t1, X, y)
    t0 -= alpha * g0                    # theta_j <- theta_j - alpha*dJ/dtheta_j
    t1 -= alpha * g1

print(round(t0, 3), round(t1, 3))
```

Perceptrón con NumPy

```python
import numpy as np

def escalon(z):                          # activacion
    return np.where(z > 0, 1, 0)

def perceptron_fit(X, y, alpha=1.0, max_epocas=20):
    w = np.zeros(X.shape[1])
    for epoca in range(max_epocas):
        errores = 0
        for xi, yi in zip(X, y):
            y_hat = escalon(w @ xi)
            if y_hat != yi:                          # solo si hay error
                w = w + alpha * (yi - y_hat) * xi    # regla del perceptron
                errores += 1
        if errores == 0:
            return w, epoca + 1
    return w, max_epocas

X = np.array([[1,0,0],[1,0,1],[1,1,0],[1,1,1]], dtype=float)
y = np.array([0,1,1,1], dtype=float)
print(perceptron_fit(X, y))
```

scikit-learn: Perceptron

```
1 import numpy as np
2 from sklearn.linear_model import Perceptron
3
4 X = np.array([[0,0],[0,1],[1,0],[1,1]])
5 for nombre, y in [("OR",  [0,1,1,1]),
6                  ("AND", [0,0,0,1])]:
7     clf = Perceptron(eta0=1.0, max_iter=50, random_state=0)
8     clf.fit(X, y)
9     print(nombre, clf.coef_, clf.intercept_, clf.score(X, y))
```

`coef_` contiene los pesos de las *features* e `intercept_` el sesgo w_0.

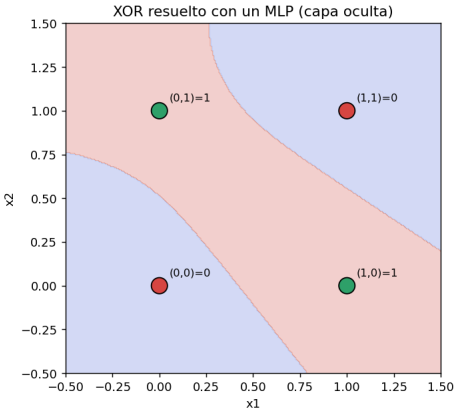
scikit-learn: XOR con un MLP

```python
import numpy as np
from sklearn.linear_model import Perceptron
from sklearn.neural_network import MLPClassifier

X = np.array([[0,0],[0,1],[1,0],[1,1]])
y = np.array([0,1,1,0])

per = Perceptron(max_iter=100).fit(X, y)
print(per.score(X, y))

mlp = MLPClassifier(hidden_layer_sizes=(4,),
                    activation="tanh", solver="
                        lbfgs",
                    max_iter=2000).fit(X, y)
print(mlp.score(X, y))
```



Resumen

- La recta puede ajustarse de forma iterativa con descenso de gradiente.
- Una neurona calcula $\mathrm{w}^\top \mathrm{x}$ y aplica una activación g.
- Con g identidad se obtiene regresión; con g escalón, el perceptrón.
- La regla $\mathrm{w} \leftarrow \mathrm{w} + \alpha(y - \hat{y})\mathrm{x}$ converge cuando los datos son linealmente separables.
- XOR no es separable; una capa oculta lo resuelve.

Referencias

- Rosenblatt, F. (1958). *The Perceptron*. Psychological Review.
- Minsky, M. & Papert, S. (1969). *Perceptrons*. MIT Press.
- Bishop, C. (2006). *Pattern Recognition and Machine Learning*. Springer, cap. 4.
- Goodfellow, I., Bengio, Y. & Courville, A. (2016). *Deep Learning*. MIT Press, cap. 6.
- scikit-learn. `https://scikit-learn.org`